



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Práctica No. 8: Aplicación de relaciones de composición entre clases.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Itzel Barriba Cázares

27/02/2026

Tabla de contenido

I.	Objetivo	2
II.	Descripción de la practica.....	2
	Lista de materiales.....	2
III.	Desarrollo de la práctica	3
	Programa 1: Modelado de Composición en una Biblioteca	3
	Clases a implementar:	3
	Tareas a realizar:.....	4
	Prueba de la Aplicación:	5
IV.	Análisis de Resultados.....	5
	Clase Main.....	5
	Clase Biblioteca	12
	Clase Libro.....	16
	Clase Pagina	19
	Programa 1 — Lavadora.....	¡Error! Marcador no definido.
	Programa 2 — Factura.....	¡Error! Marcador no definido.
	Programa 4 — Empleado.....	¡Error! Marcador no definido.
V.	Conclusiones	21
VI.	Bibliografía.....	29
	Entrega.....	30
	Criterios para evaluar:	30
	Código (70 puntos)	30
	Reporte (30 puntos):	30

I. Objetivo

Diseñar clases, donde se utilicen relaciones de composición entre ellas, aplicando multiplicidad para representar entidades del mundo real, aplicando las capacidades de un lenguaje de programación orientada a objetos, con una actitud analítica, propositiva y con creatividad.

II. Descripción de la practica

Lista de materiales

- Computadora Personal (PC)
- Ambiente de desarrollo. (Vscode)
- JDK Java Development KitTexto.

III. Desarrollo de la práctica

Programa 1: Modelado de Composición en una Biblioteca

Se desarrollará una aplicación en Java que modele una biblioteca compuesta por libros, donde cada libro a su vez este compuesto por páginas.

Las relaciones para implementar son de composición, lo que implica que:

- Las páginas no pueden existir sin un libro.
- Los libros no pueden existir sin la biblioteca
- Si se elimina un libro, todas sus páginas se eliminan automáticamente.
- Si se elimina la biblioteca todos los libros y sus páginas desaparecen.

Restricción: Se deben utilizar únicamente arreglos, no colecciones (ArrayList, List, etc).

Clases a implementar:

Clase Biblioteca:

Representa una biblioteca que contiene múltiples libros.

Atributos:

- nombre: String: Nombre de la biblioteca.
- libros: Libro[]: Arreglo de libros.
- contadorLibros: int número actual de libros almacenados

Métodos:

- agregarLibro(Libro libro): Crea y agrega un libro al arreglo.
- eliminarLibro(Libro libro): Elimina un libro de la biblioteca y libera su referencia.
- mostrarLibros(): Muestra la información de todos los libros

Clase Libro:

Representa un libro dentro de la Biblioteca.

Atributos:

- titulo: String
- paginas: Pagina[], Arreglo de páginas.
- contadorPaginas: int

Métodos:

- agregarPagina(): crea y agrega una página.
- mostrarPaginas(): Muestra el contenido de las páginas.
- Importante: Las páginas deben crearse dentro del libro (no desde el main)

Clase Pagina:

Representa una página dentro de un libro.

Atributos:

- numero: int
- contenido: String: Contenido de la página.

Métodos:

- mostrarContenido(): Imprime el contenido de las páginas.

Tareas a realizar:

- Crear una instancia de Biblioteca
- Agregar varios libros utilizando los métodos definidos
- Para cada libro, agregar múltiples páginas con contenido simulado
- Mostrar la lista de libros y sus páginas.
- Eliminar un libro específico y verificar que:
 - El libro ya no se encuentra en el arreglo
 - Sus páginas dejan de existir
- Simular la eliminación de la biblioteca (asignando null) y analizar qué ocurre con los objetos contenidos.

Prueba de la Aplicación:

Desarrolla un programa principal que:

- Cree una biblioteca
- Agregue al menos 2 libros
- Agregue páginas a cada libro
- Muestre la información completa
- Elimine un libro
- Verifique que sus páginas también se eliminan.

IV. Análisis de Resultados

Clase Main

Se recomienda mejor abrir los archivos adjuntados en “src.zip” para una mejor visualización.

```

/*****
 * File: Main.java
 * Author: Joshua Osorio Osorio
 * Materia: Programación Orientada a Objetos (POO)
 * Date: 01/May/2026
 * Comments: Programa principal para probar la composición entre
 * Biblioteca, Libro y Pagina. Incluye menú interactivo para
 * probar todas las funcionalidades.
 *****/

import java.util.Scanner;

public class Main {
    /* Biblioteca principal que se usará en todo el programa */
    private static Biblioteca biblioteca;

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int opcion;

        /* Inicializar biblioteca por defecto */
        biblioteca = new Biblioteca("Biblioteca Central", 100);

        do {
            System.out.println("\n-----");
            System.out.println("          PRÁCTICA 8: COMPOSICIÓN - BIBLIOTECA          ");
            System.out.println("-----");
            System.out.println("\t1 - Agregar libro");
            System.out.println("\t2 - Agregar páginas a un libro");
            System.out.println("\t3 - Mostrar todos los libros");
            System.out.println("\t4 - Eliminar un libro");
            System.out.println("\t5 - Mostrar información de la biblioteca");
            System.out.println("\t6 - Crear nueva biblioteca (capacidad personalizada)");
            System.out.println("\t7 - Demostración automática (prueba completa)");
            System.out.println("\t8 - Salir");
            System.out.print("\n\tOpción: ");
            opcion = input.nextInt();
            input.nextLine(); /* Limpiar buffer */

            switch (opcion) {
                case 1:
                    agregarLibro(input);
                    break;
                case 2:
                    agregarPaginas(input);
                    break;
                case 3:
                    mostrarLibros();
                    break;
                case 4:
                    eliminarLibro(input);
                    break;
                case 5:
                    mostrarInformacionBiblioteca();
                    break;
            }
        } while (opcion != 8);
    }
}

```

```

        case 6:
            crearNuevaBiblioteca(input);
            break;
        case 7:
            demostracionAutomatica();
            break;
        case 8:
            System.out.println("\n;Hasta luego!");
            break;
        default:
            System.out.println("Opción no válida. Intente de nuevo.");
    }
} while (opcion != 8);

input.close();
}

/* ----- Opción 1 ----- */

/**
 * Agrega un nuevo libro a la biblioteca.
 * Solicita al usuario el título del libro.
 */
public static void agregarLibro(Scanner input) {
    System.out.println("\n--- AGREGAR LIBRO ---");
    System.out.println("Título del libro: ");
    String titulo = input.nextLine();

    Libro nuevoLibro = new Libro(titulo);
    boolean agregado = biblioteca.agregarLibro(nuevoLibro);

    if (agregado) {
        System.out.println("\n;Desea agregar páginas a este libro ahora? (s/n)");
        String respuesta = input.nextLine();
        if (respuesta.equalsIgnoreCase("s")) {
            agregarPaginasALibro(input, nuevoLibro);
        }
    }
}

/* ----- Opción 2 ----- */

/**
 * Agrega páginas a un libro existente.
 * Primero muestra los libros disponibles y luego pide elegir uno.
 */
public static void agregarPaginas(Scanner input) {
    System.out.println("\n--- AGREGAR PÁGINAS ---");

    if (biblioteca.getContadorLibros() == 0) {
        System.out.println("No hay libros en la biblioteca. Primero agregue un libro.");
        return;
    }

    /* Mostrar libros disponibles */

```

```

        System.out.println("Libros disponibles:");
        for (int i = 0; i < biblioteca.getContadorLibros(); i++) {
            System.out.printf("  [%d] %s\n", i + 1, biblioteca.getLibros()[i].getTitulo());
        }

        System.out.print("\nSeleccione el número del libro: ");
        int seleccion = input.nextInt();
        input.nextLine();

        if (seleccion < 1 || seleccion > biblioteca.getContadorLibros()) {
            System.out.println("Selección no válida.");
            return;
        }

        Libro libroSeleccionado = biblioteca.getLibros()[seleccion - 1];
        agregarPaginasALibro(input, libroSeleccionado);
    }

    /**
     * Agrega múltiples páginas a un libro específico.
     *
     * @param input Scanner para entrada de usuario
     * @param libro Libro al que se agregarán páginas
     */
    public static void agregarPaginasALibro(Scanner input, Libro libro) {
        System.out.printf("\nAgregando páginas a: %s\n", libro.getTitulo());
        System.out.print("¿Cuántas páginas desea agregar? ");
        int cantidad = input.nextInt();
        input.nextLine();

        for (int i = 1; i <= cantidad; i++) {
            System.out.printf("Contenido de la página %d: ", i);
            String contenido = input.nextLine();
            libro.agregarPagina(contenido);
        }

        System.out.printf("\nSe agregaron %d páginas a '%s'!\n", cantidad, libro.getTitulo());
    }

    /* ----- Opción 3 ----- */

    /**
     * Muestra todos los libros y sus páginas.
     */
    public static void mostrarLibros() {
        System.out.println("\n--- MOSTRAR LIBROS ---");
        biblioteca.mostrarLibros();
    }

    /* ----- Opción 4 ----- */

    /**
     * Elimina un libro de la biblioteca.
     * Muestra los libros disponibles y pide seleccionar uno.
     */

```



```

public static void eliminarLibro(Scanner input) {
    System.out.println("\n--- ELIMINAR LIBRO ---");

    if (biblioteca.getContadorLibros() == 0) {
        System.out.println("No hay libros en la biblioteca para eliminar.");
        return;
    }

    /* Mostrar libros disponibles */
    System.out.println("Libros disponibles:");
    for (int i = 0; i < biblioteca.getContadorLibros(); i++) {
        System.out.printf(" [%d] %s\n", i + 1, biblioteca.getLibros()[i].getTitulo());
    }

    System.out.print("\nSeleccione el número del libro a eliminar: ");
    int seleccion = input.nextInt();
    input.nextLine();

    if (seleccion < 1 || seleccion > biblioteca.getContadorLibros()) {
        System.out.println("Selección no válida.");
        return;
    }

    Libro libroEliminar = biblioteca.getLibros()[seleccion - 1];
    biblioteca.eliminarLibro(libroEliminar);
}

/* ----- Opción 5 ----- */

/**
 * Muestra información estadística de la biblioteca.
 */
public static void mostrarInformacionBiblioteca() {
    System.out.println("\n--- INFORMACIÓN DE LA BIBLIOTECA ---");
    System.out.printf("Nombre: %s\n", biblioteca.getNombre());
    System.out.printf("Capacidad total: %d libros\n", biblioteca.getCapacidad());
    System.out.printf("Libros actuales: %d\n", biblioteca.getContadorLibros());
    System.out.printf("Espacio disponible: %d libros\n", biblioteca.espacioDisponible());
    System.out.printf("¿Está vacía? %s\n", biblioteca.estaVacía() ? "Sí" : "No");
    System.out.printf("¿Está llena? %s\n", biblioteca.estaLlena() ? "Sí" : "No");
    /* Los operadores ternarios tambien se pueden usar como en python */
}

/* ----- Opción 6 ----- */

/**
 * Crea una nueva biblioteca con capacidad personalizada.
 */
public static void crearNuevaBiblioteca(Scanner input) {
    System.out.println("\n--- CREAR NUEVA BIBLIOTECA ---");
    System.out.print("Nombre de la nueva biblioteca: ");
    String nombre = input.nextLine();
    System.out.print("Capacidad máxima de libros: ");
    int capacidad = input.nextInt();
    input.nextLine();
}

```

```

    if (capacidad <= 0) {
        System.out.println("Capacidad inválida. Se usará capacidad = 10.");
        capacidad = 10;
    }

    biblioteca = new Biblioteca(nombre, capacidad);
    System.out.printf("\n;Nueva biblioteca '%s' creada con capacidad de %d libros!\n",
        nombre, capacidad);
}

/* ----- Opción 7 ----- */

/**
 * Demostración automática que prueba todas las funcionalidades.
 * No requiere intervención del usuario.
 */
public static void demostracionAutomatica() {
    System.out.println("\n-----");
    System.out.println("                DEMOSTRACIÓN AUTOMATICA COMPLETA                ");
    System.out.println("-----\n");

    /* 1. Crear biblioteca de prueba */
    System.out.println(">>> 1. Creando biblioteca 'Biblioteca de Prueba' con capacidad de 4 libros");
    Biblioteca bibliotecaDemo = new Biblioteca("Biblioteca de Prueba", 4);
    bibliotecaDemo.mostrarLibros();

    /* 2. Crear libros y agregar páginas */
    System.out.println("\n>>> 2. Creando libros y agregando páginas");

    /* Dedicado a mi florecita loca */
    Libro libroA = new Libro("El Principito");
    libroA.agregarPagina("Primero te sentarás en la hierba, un poco retirado de mí,");
    libroA.agregarPagina("yo te miraré de reojo y tú no dirás nada.");
    libroA.agregarPagina("Las palabras son fuentes de malentendidos.");

    Libro libroB = new Libro("El arte de amar"); /* Erich Fromm */
    libroB.agregarPagina("En primer lugar, la práctica de un arte requiere disciplina...");
    libroB.agregarPagina("Un tercer factor es la paciencia. Repetimos que quien haya...");
    libroB.agregarPagina("ahorrrarlo lo matamos, porque no sabemos que hacer con el");

    Libro libroC = new Libro("La caída"); /* Albert Camus */
    libroC.agregarPagina("Unos gritan: <<¡ámame!>>; otros, <<¡no me ames!>>.");
    libroC.agregarPagina("Pero cierta clase de hombres, la peor y la más infeliz:");
    libroC.agregarPagina("<<¡no me ames y guárdame fidelidad!>>");

    Libro libroD = new Libro("El amor a la vida"); /* Erich Fromm */
    libroD.agregarPagina("Hacemos de todo para no perder tiempo,");
    libroD.agregarPagina("Para ahorrrarlo, y cuando hemos logrado salvarlo o ");
    libroD.agregarPagina("Si el arte no es algo de suprema importancia, el aprendiz jamás lo\r\n" + //
        "dominará.");

```

```

/* 3. Agregar libros a la biblioteca */
System.out.println("\n>>> 3. Agregando libros a la biblioteca");
bibliotecaDemo.agregarLibro(libroA);
bibliotecaDemo.agregarLibro(libroB);
bibliotecaDemo.agregarLibro(libroC);
bibliotecaDemo.agregarLibro(libroD);

/* Intentar agregar un quinto libro (capacidad = 4, debe fallar) */
Libro libroExtra = new Libro("Libro Extra (no cabe)");
libroExtra.agregarPagina("Este libro no debería poder agregarse...");
System.out.println("\n>>> Probando límite de capacidad (debe fallar):");
bibliotecaDemo.agregarLibro(libroExtra);

/* 4. Mostrar todos los libros */
System.out.println("\n>>> 4. Mostrando todos los libros de la biblioteca");
bibliotecaDemo.mostrarLibros();

/* 5. Mostrar información estadística */
System.out.println("\n>>> 5. Información estadística");
System.out.printf("  Nombre: %s\n", bibliotecaDemo.getNombre());
System.out.printf("  Capacidad: %d\n", bibliotecaDemo.getCapacidad());
System.out.printf("  Libros actuales: %d\n", bibliotecaDemo.getContadorLibros());
System.out.printf("  Espacio disponible: %d\n", bibliotecaDemo.espacioDisponible());
System.out.printf("  ¿Está llena? %s\n", bibliotecaDemo.estaLlena() ? "Sí" : "No");

/* 6. Eliminar un libro */
System.out.println("\n>>> 6. Eliminando 'El Principito'");
bibliotecaDemo.eliminarLibro(libroA);

/* 7. Mostrar después de eliminar */
System.out.println("\n>>> 7. Biblioteca después de eliminar el libro");
bibliotecaDemo.mostrarLibros();

/* 8. Agregar un libro nuevo al espacio liberado */
System.out.println("\n>>> 8. Agregando un libro nuevo al espacio disponible");
Libro libroNuevo = new Libro("Cien años de soledad");
libroNuevo.agregarPagina("Muchos años después, frente al pelotón de fusilamiento...");
libroNuevo.agregarPagina("el coronel Aureliano Buendía había de recordar aquella tarde
remota...");
bibliotecaDemo.agregarLibro(libroNuevo);

/* 9. Mostrar estado final */
System.out.println("\n>>> 9. Estado final de la biblioteca");
bibliotecaDemo.mostrarLibros();

```

UABC_FCQI_PROGRAMACIÓN_ORIENTAOBJETOS.

```
/* 10. Simular eliminación de biblioteca */
System.out.println("\n>>> 10. Simulando eliminación de la biblioteca (bibliotecaDemo =
null)");
System.out.println("    Nota: Los objetos Libro aún existen porque los variables");
System.out.println("    libroA, libroB, libroC, libroExtra, libroNuevo los referen-
cian.");
System.out.println("    Sin esas referencias externas, todos los objetos (biblioteca,");
System.out.println("    libros y páginas) quedarían elegibles para el Garbage Collec-
tor.\n");

System.out.println("-----");
System.out.println("                    DEMOSTRACIÓN COMPLETADA                    ");
System.out.println("-----\n");
}
```

Clase Biblioteca

```

/*****
 * File: Biblioteca.java
 * Author: Joshua Osorio Osorio
 * Materia: Programación Orientada a Objetos (POO)
 * Date: 01/May/2026
 * Comments: Clase que representa una biblioteca compuesta por múltiples libros.
 * La relación con Libro es de composición: los libros no existen sin la
 * biblioteca.
 *****/

public class Biblioteca {
    private int capacidad; /* Capacidad máxima de la biblioteca */
    private String nombre;
    private Libro[] libros;
    private int contadorLibros;

    /**
     * Constructor por defecto (capacidad = 100)
     */
    public Biblioteca() {
        this.nombre = "Biblioteca sin nombre";
        this.capacidad = 100;
        this.libros = new Libro[capacidad];
        this.contadorLibros = 0;
    }

    /**
     * Constructor con nombre (capacidad por defecto = 100)
     *
     * @param nombre Nombre de la biblioteca
     */
    public Biblioteca(String nombre) {
        this.nombre = nombre;
        this.capacidad = 100;
        this.libros = new Libro[capacidad];
        this.contadorLibros = 0;
    }

    /**
     * Constructor principal con nombre y capacidad personalizada.
     *
     * @param nombre Nombre de la biblioteca
     * @param capacidad Capacidad máxima de libros (debe ser mayor a 0)
     */
    public Biblioteca(String nombre, int capacidad) {
        if (capacidad <= 0) {
            System.out.println("La capacidad debe ser mayor a 0. Se usará capacidad = 10 por de-
fecto.");
            capacidad = 10; /* Valor por defecto */
        }
        this.nombre = nombre;
        this.capacidad = capacidad;
        this.libros = new Libro[capacidad];
        this.contadorLibros = 0;
    }
}

```

```

/* ----- Setters ----- */

public void setNombre(String nombre) {
    this.nombre = nombre;
}

/**
 * Definir arreglo de libros completo (util para restauración).
 *
 * @param libros   Arreglo de libros
 * @param contador Cuántos libros tiene actualmente
 */
public void setLibros(Libro[] libros, int contador) {
    if (libros == null) {
        System.out.println("El arreglo de libros no puede ser null");
        return; /* Salir del metodo */
    }
    if (contador < 0 || contador > libros.length) {
        System.out.println("Contador inválido");
        return; /* Salir del metodo */
    }
    this.libros = libros;
    this.contadorLibros = contador;
    this.capacidad = libros.length;
}

/* ----- Getters ----- */

public String getNombre() {
    return nombre;
}

public int getContadorLibros() {
    return contadorLibros;
}

public int getCapacidad() {
    return capacidad;
}

public Libro[] getLibros() {
    return libros;
}

/**
 * Agrega un libro al arreglo si hay capacidad.
 *
 * @param libro Libro a agregar (no puede ser null)
 * @return true si se agregó, false si la biblioteca está llena
 */
public boolean agregarLibro(Libro libro) {
    if (libro == null) {
        System.out.println("Error: no se puede agregar un libro nulo.");
        return false;
    }
}

```

```

        if (contadorLibros < capacidad) {
            libros[contadorLibros++] = libro;
            System.out.printf("Libro '%s' agregado correctamente. (%d/%d)\n",
                libro.getTitulo(), contadorLibros, capacidad);
            return true;
        } else {
            System.out.printf("Error: la biblioteca esta llena (capacidad: %d). No se puede agre-
gar '%s'.\n",
                capacidad, libro.getTitulo());
            return false;
        }
    }

    /**
     * Elimina un libro de la biblioteca por referencia.
     * Realiza desplazamiento y libera la última posición.
     *
     * @param libro Libro a eliminar
     * @return true si se eliminó, false si no se encontró
     */
    public boolean eliminarLibro(Libro libro) {
        if (libro == null) {
            System.out.println("Error: no se puede eliminar un libro nulo.");
            return false;
        }

        int indice = -1;
        for (int i = 0; i < contadorLibros; i++) {
            if (libros[i] == libro) {
                indice = i;
                break;
            }
        }

        if (indice == -1) {
            System.out.printf("Error: el libro '%s' no se encuentra en la biblioteca.\n",
                libro.getTitulo());
            return false;
        }

        String tituloEliminado = libros[indice].getTitulo();

        /* Desplazar los elementos a la izquierda */
        for (int i = indice; i < contadorLibros - 1; i++) {
            libros[i] = libros[i + 1];
        }
        libros[--contadorLibros] = null; /* liberar referencia */

        System.out.printf("Libro '%s' eliminado correctamente. (%d/%d libros restantes)\n",
            tituloEliminado, contadorLibros, capacidad);
        return true;
    }
}

```

```

/**
 * Muestra toda la información de la biblioteca:
 * nombre, capacidad, lista de libros y el contenido de sus páginas.
 */
public void mostrarLibros() {
    System.out.printf("\n-----\n");
    System.out.printf(" BIBLIOTECA: %-40s |\n", nombre);
    System.out.printf(" Capacidad: %d libros | Ocupados: %d\n",
        capacidad, contadorLibros);
    System.out.printf("-----\n");

    if (contadorLibros == 0) {
        System.out.println("No hay libros en la biblioteca.\n");
        return;
    }

    System.out.println("\nLISTA DE LIBROS:");
    System.out.println("-----");
    for (int i = 0; i < contadorLibros; i++) {
        System.out.printf("\n[%d] Titulo: %s\n", i + 1, libros[i].getTitulo());
        libros[i].mostrarPaginas();
    }
    System.out.println("\n-----");
}

/**
 * Verifica si la biblioteca está vacía.
 *
 * @return true si no hay libros
 */
public boolean estaVacía() {
    return contadorLibros == 0;
}

/**
 * Verifica si la biblioteca está llena.
 *
 * @return true si alcanzó la capacidad máxima
 */
public boolean estaLlena() {
    return contadorLibros >= capacidad;
}

/**
 * Espacio disponible para más libros.
 *
 * @return número de espacios libres
 */
public int espacioDisponible() {
    return capacidad - contadorLibros;
}
}

```

Clase Libro


```

/*****
 * File: Libro.java
 * Author: Joshua Osorio Osorio
 * Materia: Programación Orientada a Objetos (POO)
 * Date: 01/May/2026
 * Comments: Clase que representa un libro dentro de una biblioteca.
 * Contiene un arreglo de páginas, las cuales se crean exclusivamente
 * dentro de esta clase (composición fuerte).
 *****/

public class Libro {
    private int maxPaginas; /* Capacidad máxima de páginas variable */
    private String titulo;
    private Pagina[] paginas;
    private int contadorPaginas;

    /**
     * Constructor vacío (título por defecto "Sin título").
     * Capacidad por defecto: 500 páginas
     */
    public Libro() {
        this("Sin título", 500); /* Reutiliza el constructor principal */
    }

    /**
     * Constructor que inicializa el libro con un título.
     * Capacidad por defecto: 500 páginas
     *
     * @param titulo Título del libro
     */
    public Libro(String titulo) {
        this(titulo, 500);
    }

    /**
     * Constructor principal con título y capacidad personalizada.
     *
     * @param titulo Título del libro
     * @param maxPaginas Capacidad máxima de páginas (debe ser mayor a 0)
     */
    public Libro(String titulo, int maxPaginas) {
        if (maxPaginas <= 0) {
            System.out.println("La capacidad de páginas debe ser mayor a 0. Se usará 100.");
            maxPaginas = 100;
        }
        this.titulo = titulo;
        this.maxPaginas = maxPaginas;
        this.paginas = new Pagina[maxPaginas];
        this.contadorPaginas = 0;
    }
}

```

```

/* ----- Getters ----- */

public String getTitulo() {
    return titulo;
}

public int getContadorPaginas() {
    return contadorPaginas;
}

public int getMaxPaginas() {
    return maxPaginas;
}

/* ----- Setters ----- */

public void setTitulo(String titulo) {
    this.titulo = titulo;
}

/**
 * Agrega una nueva página al libro. La página se crea internamente
 * con el número de página automático (contadorPaginas + 1).
 *
 * @param contenido Texto de la página
 * @return true si se pudo agregar, false si el libro está lleno
 */
public boolean agregarPagina(String contenido) {
    if (contadorPaginas < maxPaginas) {
        paginas[contadorPaginas] = new Pagina(contadorPaginas + 1, contenido);
        contadorPaginas++;
        System.out.printf("  Página %d agregada a '%s' (%d/%d páginas)\n",
            contadorPaginas, titulo, contadorPaginas, maxPaginas);
        return true;
    } else {
        System.out.printf("  Error: el libro '%s' ha alcanzado su máximo de %d páginas.\n",
            titulo, maxPaginas);
        return false;
    }
}

/**
 * Muestra el contenido de todas las páginas del libro.
 */
public void mostrarPaginas() {
    if (contadorPaginas == 0) {
        System.out.println("  (El libro no tiene páginas)");
        return;
    }
    for (int i = 0; i < contadorPaginas; i++) {
        paginas[i].mostrarContenido();
    }
}

```

```
/**
 * Verifica si el libro está lleno (no admite más páginas).
 *
 * @return true si alcanzó la capacidad máxima
 */
public boolean estaLleno() {
    return contadorPaginas >= maxPaginas;
}

/**
 * Espacio disponible para más páginas.
 *
 * @return número de páginas que aún se pueden agregar
 */
public int espacioDisponible() {
    return maxPaginas - contadorPaginas;
}
}
```

Clase Pagina

```

/*****
 * File: Pagina.java
 * Author: Joshua Osorio Osorio
 * Materia: Programación Orientada a Objetos (POO)
 * Date: 01/May/2026
 * Comments: Clase que representa una página perteneciente a un libro.
 * No puede existir sin su libro (composición).
 *****/

public class Pagina {
    private int numero; /* Número de la página (empieza en 1) */
    private String contenido; /* Texto o contenido de la página */

    /**
     * Constructor que crea una página con número y contenido.
     * Normalmente es invocado solo desde Libro.agregarPagina().
     *
     * @param numero    Número de página (empezando en 1)
     * @param contenido Texto de la página
     */
    public Pagina(int numero, String contenido) {
        this.numero = numero;
        this.contenido = contenido;
    }

    /** Constructor por omisión (número 0, contenido vacío). */
    public Pagina() {
        this(0, ""); /* Se utiliza para llamar al constructor anterior y no escribir tanto código */
    }

    /* ----- Getters ----- */

    public int getNumero() {
        return numero;
    }

    public String getContenido() {
        return contenido;
    }

    /* ----- Setters ----- */

    public void setNumero(int numero) {
        this.numero = numero;
    }

    public void setContenido(String contenido) {
        this.contenido = contenido;
    }

    /* ----- Métodos ----- */
    /** Imprime la página en formato: " Página X: contenido" */
    public void mostrarContenido() {
        System.out.printf("  Página %d: %s\n", numero, contenido);
    }
}

```

Salida en terminal

Copilación y ejecución.

```

PowerShell 7.6.1
Página de códigos activa: 65001
Loading personal and system profiles took 848ms.
> cd "uabc.edu.mx\8_Semestre\36282_ProgramacinOrientadaAObjetos\2_Laboratorio\
L08_Tema"
> ls

Directory: uabc.edu.mx\8_Semestre\36282_ProgramacinOrientadaAObjetos\2
_Laboratorio\L08_Tema

Mode                LastWriteTime         Length Name
----                -
lar--      08/05/2026 01:52 p. m.             docs
lar--      01/05/2026 11:35 p. m.             output
l-----      01/05/2026 10:14 p. m.             src
la----- 11/03/2026 12:35 a. m.             0 README.md

> javac -d output -sourcepath src src/*.java
> java -cp output Main

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 7

```

1 - Agregar libro

```

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 1

— AGREGAR LIBRO —
Título del libro:
Los cinco lenguajes del amor
Libro 'Los cinco lenguajes del amor' agregado correctamente. (1/100)

¿Desea agregar páginas a este libro ahora? (s/n)
s

Agregando páginas a: Los cinco lenguajes del amor
¿Cuántas páginas desea agregar? 1
Contenido de la página 1: EL amor comienza, o debe comenzar, en el hogar.
Página 1 agregada a 'Los cinco lenguajes del amor' (1/500 páginas)

¡Se agregaron 1 páginas a 'Los cinco lenguajes del amor'!

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 1

— AGREGAR LIBRO —
Título del libro:

Libro '' agregado correctamente. (2/100)

¿Desea agregar páginas a este libro ahora? (s/n)
n

```

2 - Agregar páginas a un libro

```

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 2

— AGREGAR P?GINAS —
Libros disponibles:
[1] Los cinco lenguajes del amor
[2]

Seleccione el número del libro: 1

Agregando páginas a: Los cinco lenguajes del amor
¿Cuántas páginas desea agregar? 1
Contenido de la página 1: Si queremos ser efectivos comunicadores del amor, debemos estar dispuestos a a
prender el lenguaje amoroso más importante de nuestro conyuge.
Página 2 agregada a 'Los cinco lenguajes del amor' (2/500 páginas)

¡Se agregaron 1 páginas a 'Los cinco lenguajes del amor'!

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 2

— AGREGAR P?GINAS —
Libros disponibles:
[1] Los cinco lenguajes del amor
[2]

Seleccione el número del libro: 2

Agregando páginas a:
¿Cuántas páginas desea agregar? 0

¡Se agregaron 0 páginas a ''!

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción:

```

3 - Mostrar todos los libros

```

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 3

— MOSTRAR LIBROS —

BIBLIOTECA: Biblioteca Central |
Capacidad: 100 libros | Ocupados: 2

LISTA DE LIBROS:

[1] Título: Los cinco lenguajes del amor
    Página 1: El amor comienza, o debe comenzar, en el hogar.
    Página 2: Si queremos ser efectivos comunicadores del amor, debemos estar dispuestos a aprender el len
    guaje amoroso más importante de nuestro conyuge.

[2] Título:
    (El libro no tiene páginas)
    
```


4 - Eliminar un libro

```

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 4

— ELIMINAR LIBRO —
Libros disponibles:
[1] Los cinco lenguajes del amor
[2]

Seleccione el número del libro a eliminar: 2
Libro '' eliminado correctamente. (1/100 libros restantes)

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: 4

— ELIMINAR LIBRO —
Libros disponibles:
[1] Los cinco lenguajes del amor

Seleccione el número del libro a eliminar: 3
Selección no válida.

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

1 - Agregar libro
2 - Agregar páginas a un libro
3 - Mostrar todos los libros
4 - Eliminar un libro
5 - Mostrar información de la biblioteca
6 - Crear nueva biblioteca (capacidad personalizada)
7 - Demostración automática (prueba completa)
8 - Salir

Opción: |

```

5 - Mostrar información de la biblioteca

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

- 1 - Agregar libro
- 2 - Agregar páginas a un libro
- 3 - Mostrar todos los libros
- 4 - Eliminar un libro
- 5 - Mostrar información de la biblioteca
- 6 - Crear nueva biblioteca (capacidad personalizada)
- 7 - Demostración automática (prueba completa)
- 8 - Salir

Opción: 5

— INFORMACI?N DE LA BIBLIOTECA —

Nombre: Biblioteca Central
Capacidad total: 100 libros
Libros actuales: 1
Espacio disponible: 99 libros
¿Está vacía? No
¿Está llena? No

6 - Crear nueva biblioteca (capacidad personalizada)

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

- 1 - Agregar libro
- 2 - Agregar páginas a un libro
- 3 - Mostrar todos los libros
- 4 - Eliminar un libro
- 5 - Mostrar información de la biblioteca
- 6 - Crear nueva biblioteca (capacidad personalizada)
- 7 - Demostración automática (prueba completa)
- 8 - Salir

Opción: 6

— CREAR NUEVA BIBLIOTECA —

Nombre de la nueva biblioteca: Filosofía

Capacidad máxima de libros: 5

¡Nueva biblioteca 'Filosofía' creada con capacidad de 5 libros!

PR?CTICA 8: COMPOSICI?N - BIBLIOTECA

- 1 - Agregar libro
- 2 - Agregar páginas a un libro
- 3 - Mostrar todos los libros
- 4 - Eliminar un libro
- 5 - Mostrar información de la biblioteca
- 6 - Crear nueva biblioteca (capacidad personalizada)
- 7 - Demostración automática (prueba completa)
- 8 - Salir

Opción: 5

— INFORMACI?N DE LA BIBLIOTECA —

Nombre: Filosofía

Capacidad total: 5 libros

Libros actuales: 0

Espacio disponible: 5 libros

¿Está vacía? Sí

¿Está llena? No

7 - Demostración automática (prueba completa)

```

DEMOSTRACIÓN AUTOMÁTICA COMPLETA

>>> 1. Creando biblioteca 'Biblioteca de Prueba' con capacidad de 4 libros

BIBLIOTECA: Biblioteca de Prueba |
Capacidad: 4 libros | Ocupados: 0

No hay libros en la biblioteca.

>>> 2. Creando libros y agregando páginas
Página 1 agregada a 'El Principito' (1/500 páginas)
Página 2 agregada a 'El Principito' (2/500 páginas)
Página 3 agregada a 'El Principito' (3/500 páginas)
Página 1 agregada a 'El arte de amar' (1/500 páginas)
Página 2 agregada a 'El arte de amar' (2/500 páginas)
Página 3 agregada a 'El arte de amar' (3/500 páginas)
Página 1 agregada a 'La caída' (1/500 páginas)
Página 2 agregada a 'La caída' (2/500 páginas)
Página 3 agregada a 'La caída' (3/500 páginas)
Página 1 agregada a 'El amor a la vida' (1/500 páginas)
Página 2 agregada a 'El amor a la vida' (2/500 páginas)
Página 3 agregada a 'El amor a la vida' (3/500 páginas)

>>> 3. Agregando libros a la biblioteca
Libro 'El Principito' agregado correctamente. (1/4)
Libro 'El arte de amar' agregado correctamente. (2/4)
Libro 'La caída' agregado correctamente. (3/4)
Libro 'El amor a la vida' agregado correctamente. (4/4)
Página 1 agregada a 'Libro Extra (no cabe)' (1/500 páginas)

>>> Probando límite de capacidad (debe fallar):
Error: la biblioteca esta llena (capacidad: 4). No se puede agregar 'Libro Extra (no cabe)'.

>>> 4. Mostrando todos los libros de la biblioteca

BIBLIOTECA: Biblioteca de Prueba |
Capacidad: 4 libros | Ocupados: 4

LISTA DE LIBROS:

[1] Título: El Principito
Página 1: Primero te sentarás en la hierba, un poco retirado de mí,
Página 2: yo te miraré de reojo y tú no dirás nada.
Página 3: Las palabras son fuentes de malentendidos.

[2] Título: El arte de amar
Página 1: En primer lugar, la práctica de un arte requiere disciplina...
Página 2: Un tercer factor es la paciencia. Repetimos que quien haya...
Página 3: ahorrarlo lo matamos, porque no sabemos que hacer con el

[3] Título: La caída
Página 1: Unos gritan: <<¡ámame!>>; otros, <<¡no me ames!>>.
Página 2: Pero cierta clase de hombres, la peor y la más infeliz:
Página 3: <<¡no me ames y guárdame fidelidad!>>

[4] Título: El amor a la vida
Página 1: Hacemos de todo para no perder tiempo,
Página 2: Para ahorrarlo, y cuando hemos logrado salvarlo o
Página 3: Si el arte no es algo de suprema importancia, el aprendiz jamás lo
dominará.

>>> 5. Información estadística
Nombre: Biblioteca de Prueba
Capacidad: 4
Libros actuales: 4
Espacio disponible: 0
¿Está llena? Si

>>> 6. Eliminando 'El Principito'
Libro 'El Principito' eliminado correctamente. (3/4 libros restantes)

>>> 7. Biblioteca después de eliminar el libro

BIBLIOTECA: Biblioteca de Prueba |
Capacidad: 4 libros | Ocupados: 3

LISTA DE LIBROS:

[1] Título: El arte de amar

```

V. Conclusiones

Relación entre clases y composición:

La práctica permitió comprender cómo funciona la relación de composición entre clases en Java, ya que la clase Biblioteca administra objetos Libro y cada Libro contiene objetos Pagina. Una de las mayores dificultades fue mantener clara la estructura y relación entre todas las clases, debido a que cada una dependía de cómo se había diseñado previamente. Esto ayudó a desarrollar una mejor organización lógica del programa y a entender cómo los objetos pueden depender unos de otros dentro de un sistema orientado a objetos.

Uso de constructores y reutilización de código:

La implementación de múltiples constructores permitió observar cómo un constructor puede llamar a otro utilizando this(), reutilizando código y evitando repetir instrucciones. Esto facilitó la inicialización de atributos con valores por defecto y personalizados, además de hacer el código más limpio y fácil de mantener. También se comprobó la importancia de validar datos dentro de los constructores para evitar estados inválidos en los objetos.

Encapsulamiento y acceso controlado:

Todas las clases implementaron encapsulamiento utilizando atributos privados y métodos getters y setters. Esto protegió la información interna de los objetos y permitió controlar la manera en que se accede o modifica cada dato. Gracias a esto, la lógica del programa se mantuvo organizada y más segura frente a modificaciones incorrectas desde otras clases.

Manejo de arreglos de objetos:

La práctica ayudó a comprender cómo trabajar con arreglos de objetos en Java, almacenando libros dentro de la biblioteca y páginas dentro de cada libro. Además, se implementaron recorridos mediante ciclos for para mostrar información, agregar elementos y eliminar objetos realizando desplazamientos dentro del arreglo. Esto reforzó el manejo de memoria y organización de estructuras dinámicas simples.

Operadores ternarios y formato de salida:

Durante la práctica también se implementaron operadores ternarios para simplificar condiciones y hacer el código más compacto y legible, por ejemplo al mostrar estados como "Sí" o "No". Asimismo, se comprobó que en Java es posible acceder y mostrar atributos de objetos dentro de arreglos de manera similar a Python, utilizando expresiones como:

```
System.out.printf("%d] %s\n", i + 1, biblioteca.getLibros()[i].getTitulo());
```

Esto permitió entender mejor cómo recorrer colecciones de objetos y mostrar información de forma estructurada.

Separación de responsabilidades:

La organización del programa en diferentes métodos dentro de Main permitió separar la lógica de interacción con el usuario de la lógica principal de las clases Biblioteca y Libro. Esto hizo que el programa fuera más modular, entendible y sencillo de mantener, siguiendo el principio de responsabilidad única en programación orientada a objetos.

VI. Bibliografía

- [1] Oracle Corporation. (2024). Java SE Documentation. Recuperado de <https://docs.oracle.com/javase/>
- [2] Deitel, P. J., & Deitel, H. M. (2016). Java: How to Program (10th ed.). Pearson Education.

Entrega

- La entrega se realizará mediante una actividad en BB. Se han de entregar los archivos fuentes desarrollados, más el reporte de la práctica adjuntos adjuntos en un fichero zip cuyo nombre esté formado por el número de práctica y nombre del alumno. Ejemplo Practica4JuanLopez.pdf
- Hacer una clase por cada ejercicio y adjuntarlos
- Entregar un reporte de práctica, formato reporte técnico y electrónico, podrá utilizar este formato y agregar el procedimiento correspondiente), que conceptos fundamentales se aplicaron en esta práctica, junto con resultados y conclusiones.

Criterios para evaluar:

Código (70 puntos)

- Debe incluir comentarios en su programa, sin llegar al uso exagerado de ellos.
- Debe utilizar sangrías en los lugares apropiados.
- Poner como encabezado del programa en comentarios, titulo, la descripción de la práctica, nombre del alumno que realizo la práctica.
- Funcionamiento correcto del programa

Reporte (30 puntos):

Portada, competencia u objetivo	2.5 puntos
Pegar Código del programa (anexar archivos .py)	2.5 puntos
Explicación completa del código y su funcionamiento	15 puntos
Resultados (análisis de resultados)	5 puntos
Conclusiones:	5 puntos

Nota: No se aceptarán trabajos fuera de la fecha de entrega o que no cumplan con las especificaciones de entrega requeridas. Los trabajos que se detecten copiados serán anulados automáticamente para ambos alumnos.